

RT-DETRv2-S

vs **YOLOX-M**

A Controlled 13-Class Object Detection Benchmark

Same dataset. Two detector families. Accuracy - Speed - Memory - Class-wise Performance.

51.67%
mAP50:95
RT-DETRv2-S

Transformer-based detector

49.40%
mAP50:95
YOLOX-M

CNN-based detector

Unified Dataset
13 Classes

Comprehensive
Evaluation

Reproducible
Benchmark

Swipe for results →

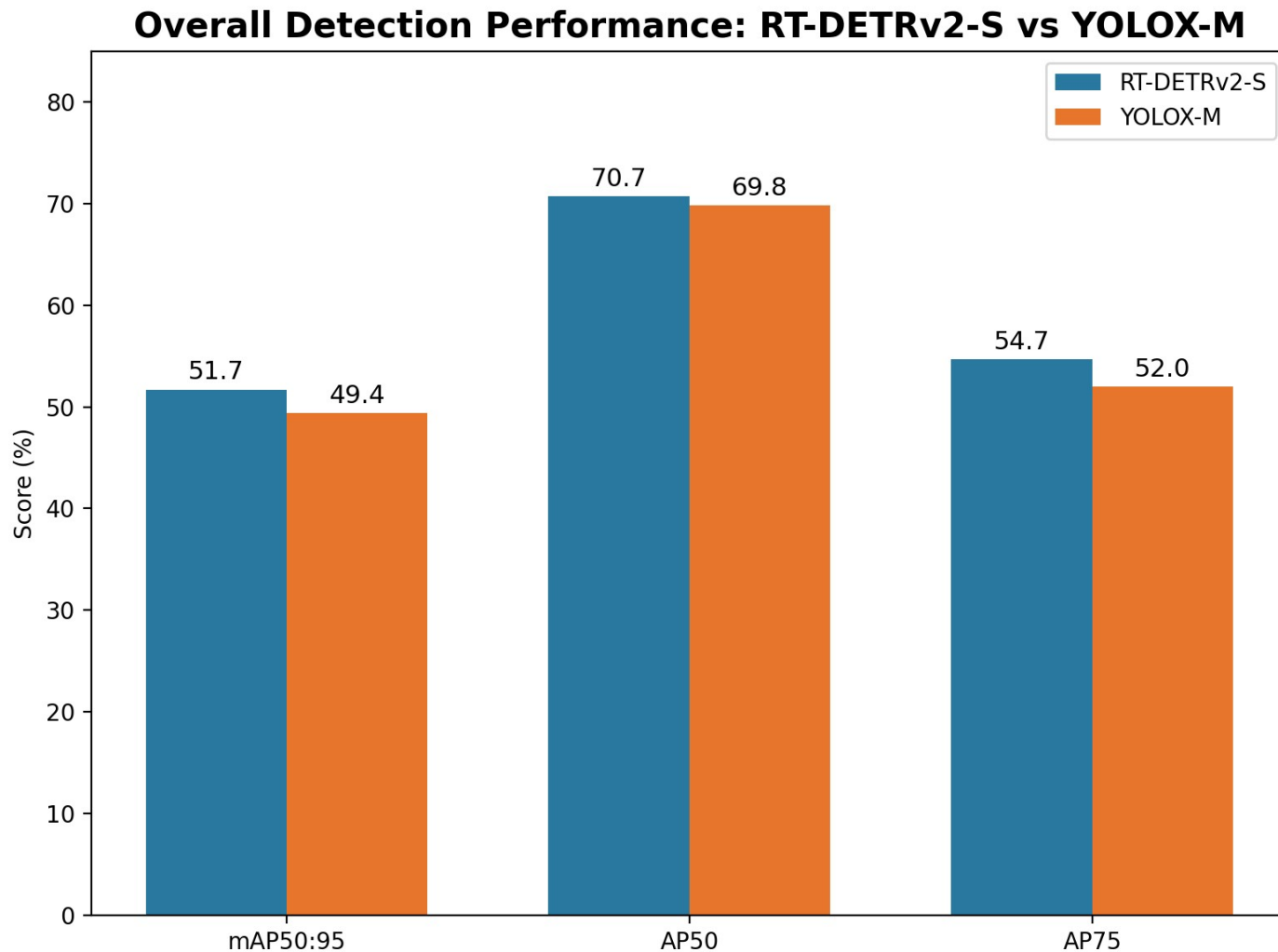
Overall detection performance

Headline metrics: mAP50:95, AP50 and AP75

KEY TAKEAWAY

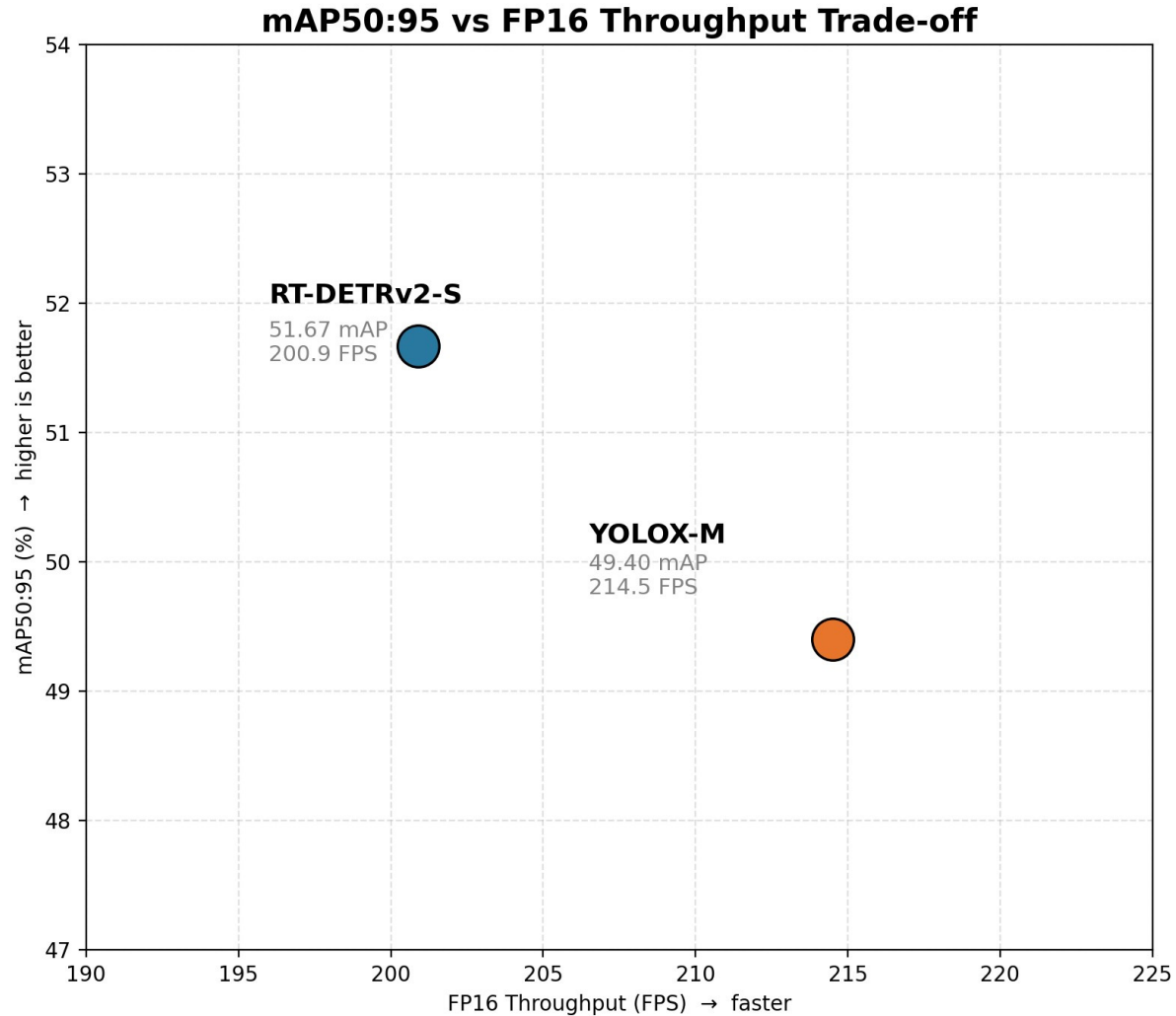
+2.27 mAP50:95 points

- RT-DETRv2-S leads across all three headline metrics.
- Higher AP75 suggests stronger localization precision.
- This is the main accuracy result for the benchmark.



mAP50:95 vs FP16 throughput

Accuracy-speed trade-off on RTX 5060 Ti



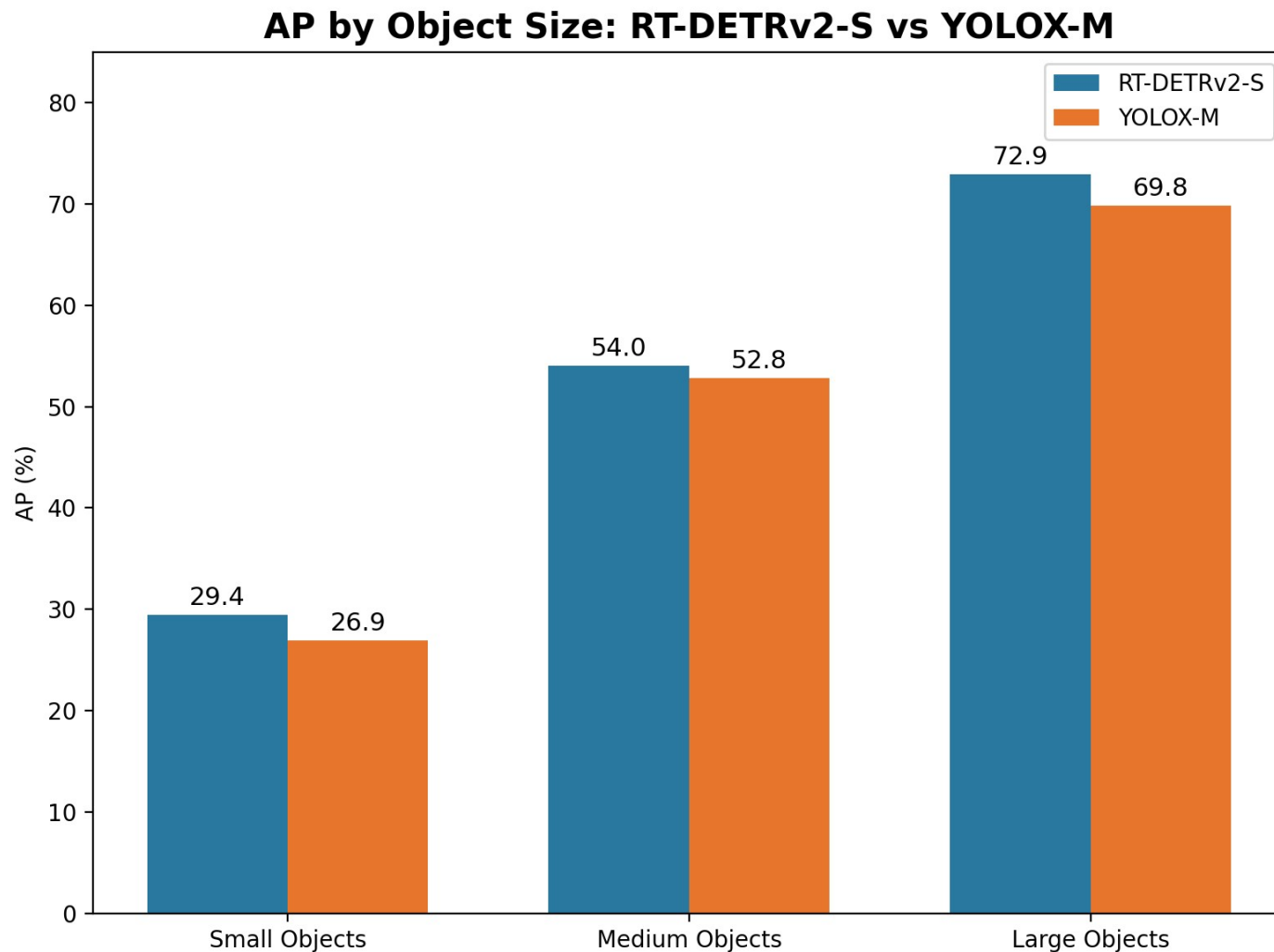
KEY TAKEAWAY

Accuracy vs speed

- RT-DETRv2-S sits higher on accuracy.
- YOLOX-M is faster in FP16 throughput.
- Choice depends on whether accuracy or maximum FPS matters more.

AP by object size

Small, medium and large-object AP



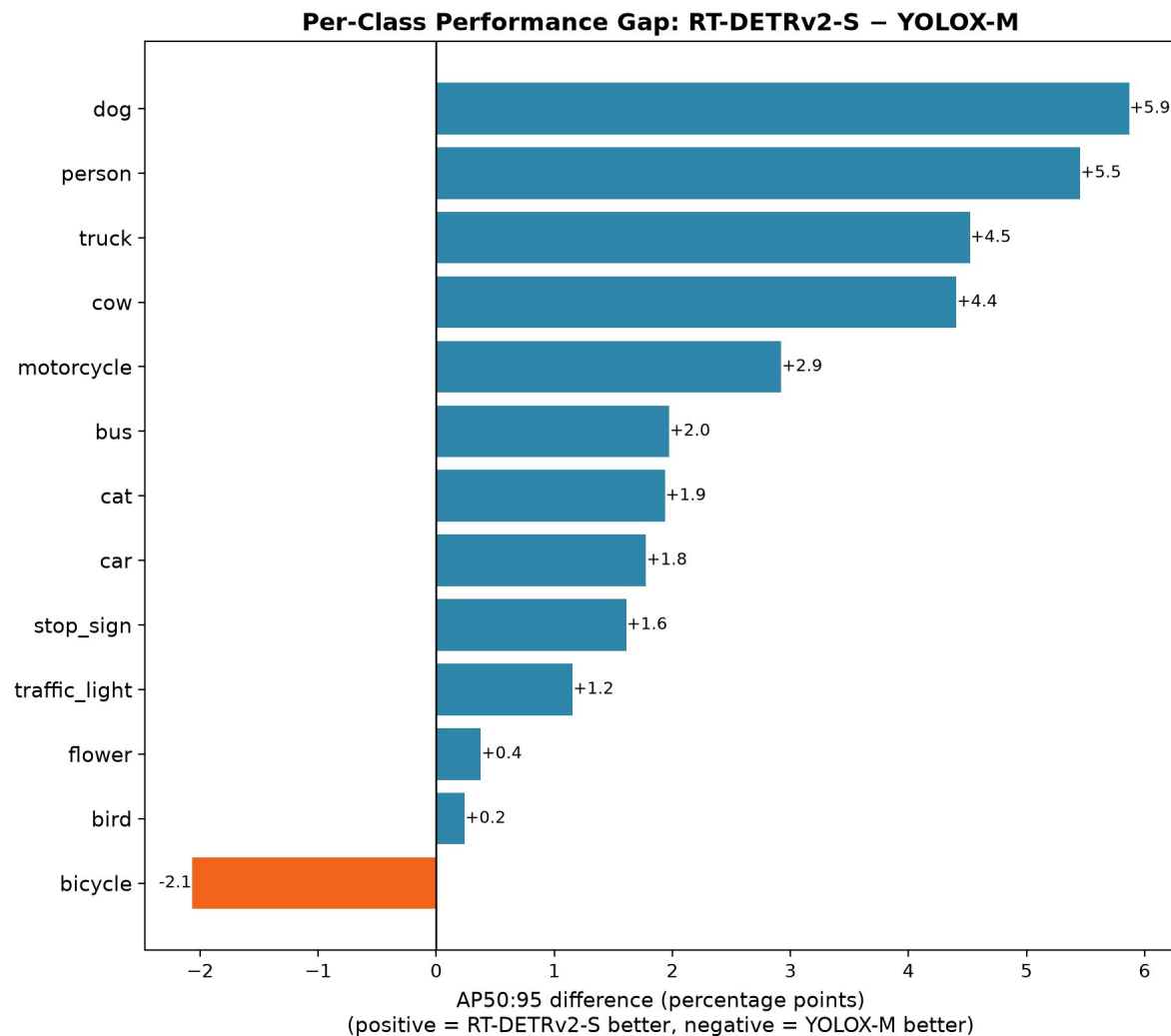
KEY TAKEAWAY

All sizes favor RT-DETRv2-S

- RT-DETRv2-S leads on small, medium and large objects.
- The largest gap appears on large-object AP.
- Small-object performance remains the hardest region.

Where did the gains come from?

RT-DETRv2-S minus YOLOX-M per-class AP50:95



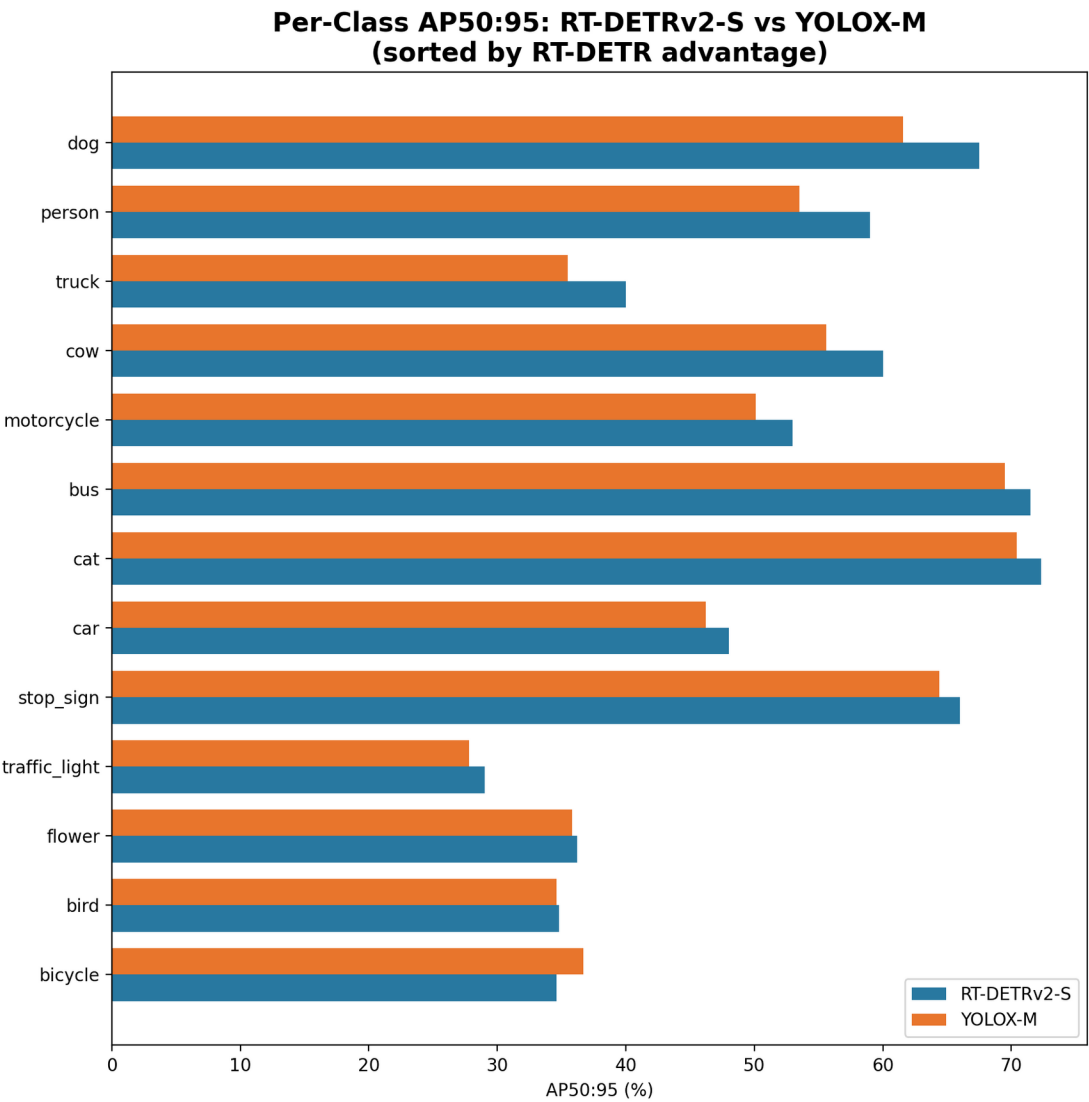
KEY TAKEAWAY

Most classes favor RT-DETRv2-S

- Largest gains: dog, person, truck and cow.
- YOLOX-M leads on bicycle in this benchmark.
- This plot explains the aggregate mAP gap.

Per-class AP50:95

Detailed AP50:95 comparison by class



KEY TAKEAWAY

Detailed class profile

- Use this slide as the class-level audit view.
- The gap chart is easier to read; this shows absolute values.
- Best for repository/report readers who want detail.



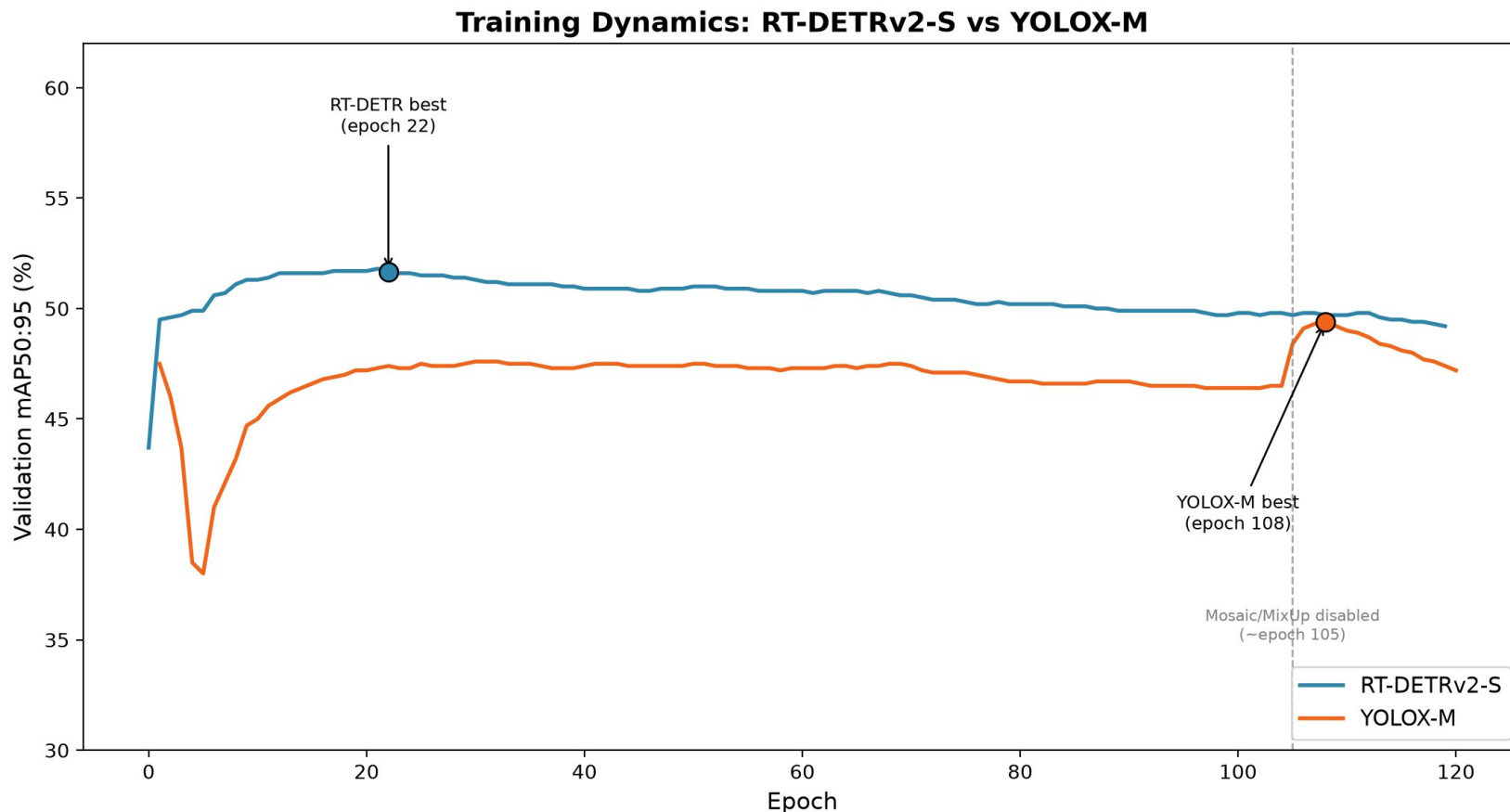
Validation mAP over training

Best checkpoints and convergence behavior

KEY TAKEAWAY

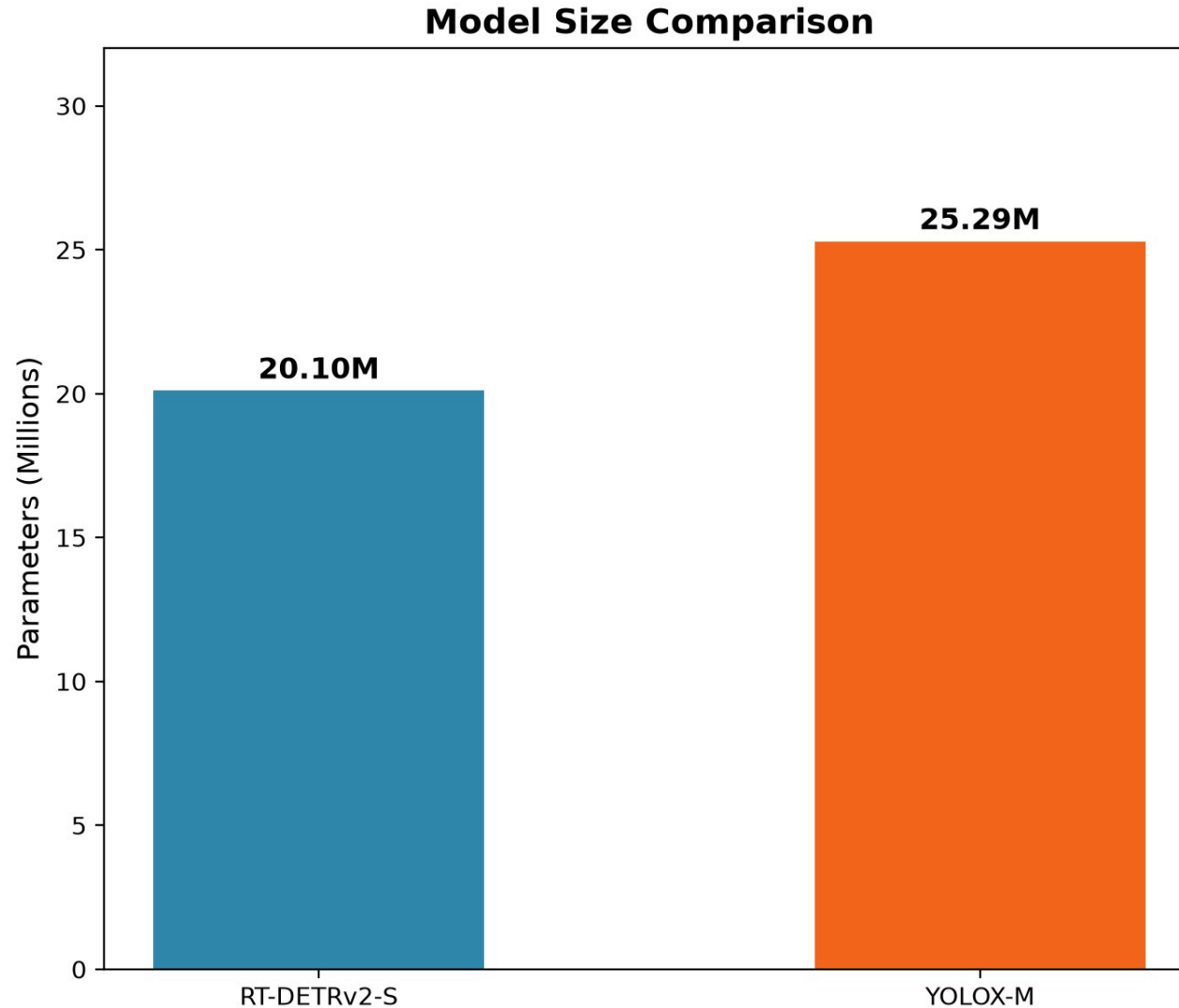
RT-DETRv2-S peaked earlier

- RT-DETRv2-S best validation checkpoint: epoch 22.
- YOLOX-M best validation checkpoint: epoch 108.
- Interpretation depends on each model's native recipe.



Parameter count comparison

Compactness matters for deployment



KEY TAKEAWAY

~20% fewer parameters

- RT-DETRv2-S: 20.10M parameters.
- YOLOX-M: 25.29M parameters.
- RT-DETRv2-S is smaller while achieving higher mAP.

FP16 speed benchmark

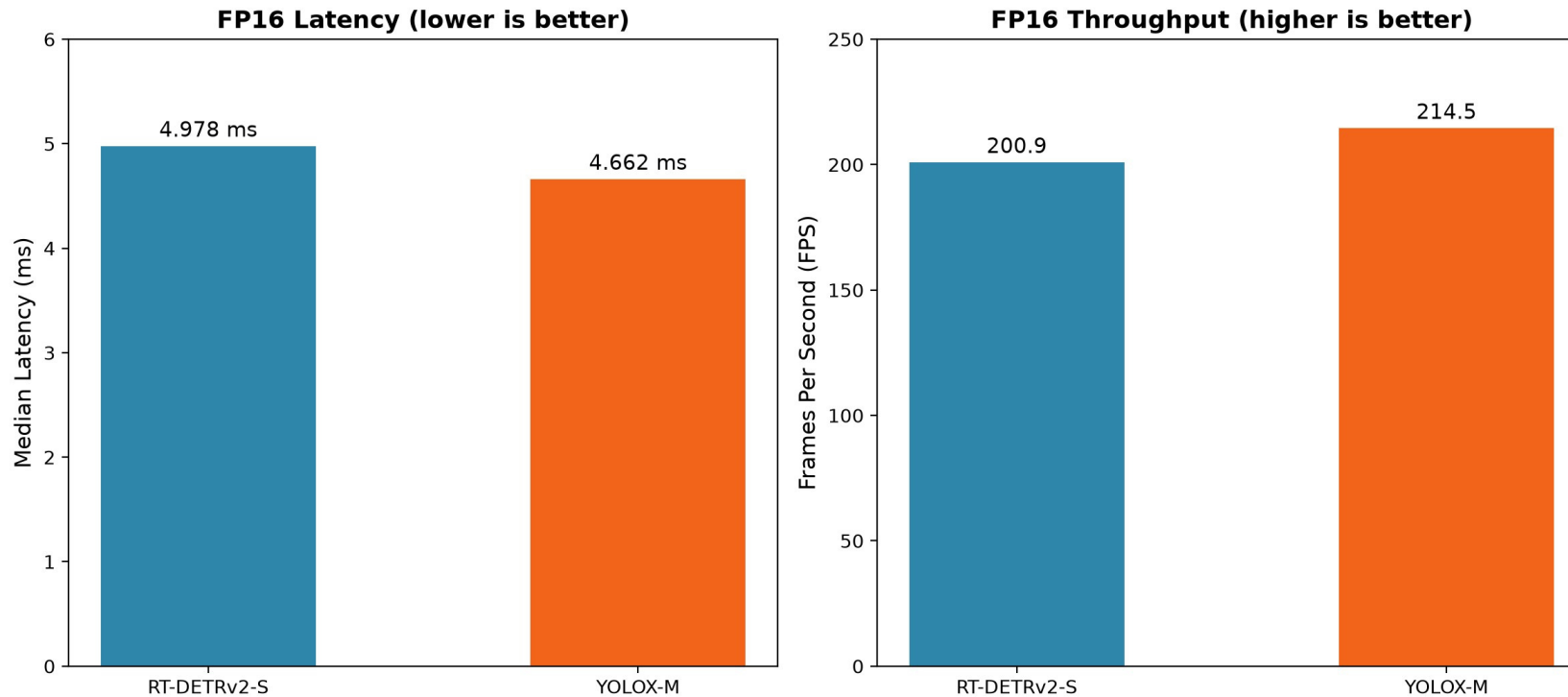
Batch=1, 640x640, RTX 5060 Ti, forward pass only

KEY TAKEAWAY

YOLOX-M is faster in FP16

- YOLOX-M: 214.5 FPS.
- RT-DETRv2-S: 200.9 FPS.
- Latency gap is modest but relevant for high-FPS deployment.

FP16 Inference Speed: RT-DETRv2-S vs YOLOX-M
(batch=1, 640×640, RTX 5060 Ti, forward-pass only)



FP32 speed benchmark

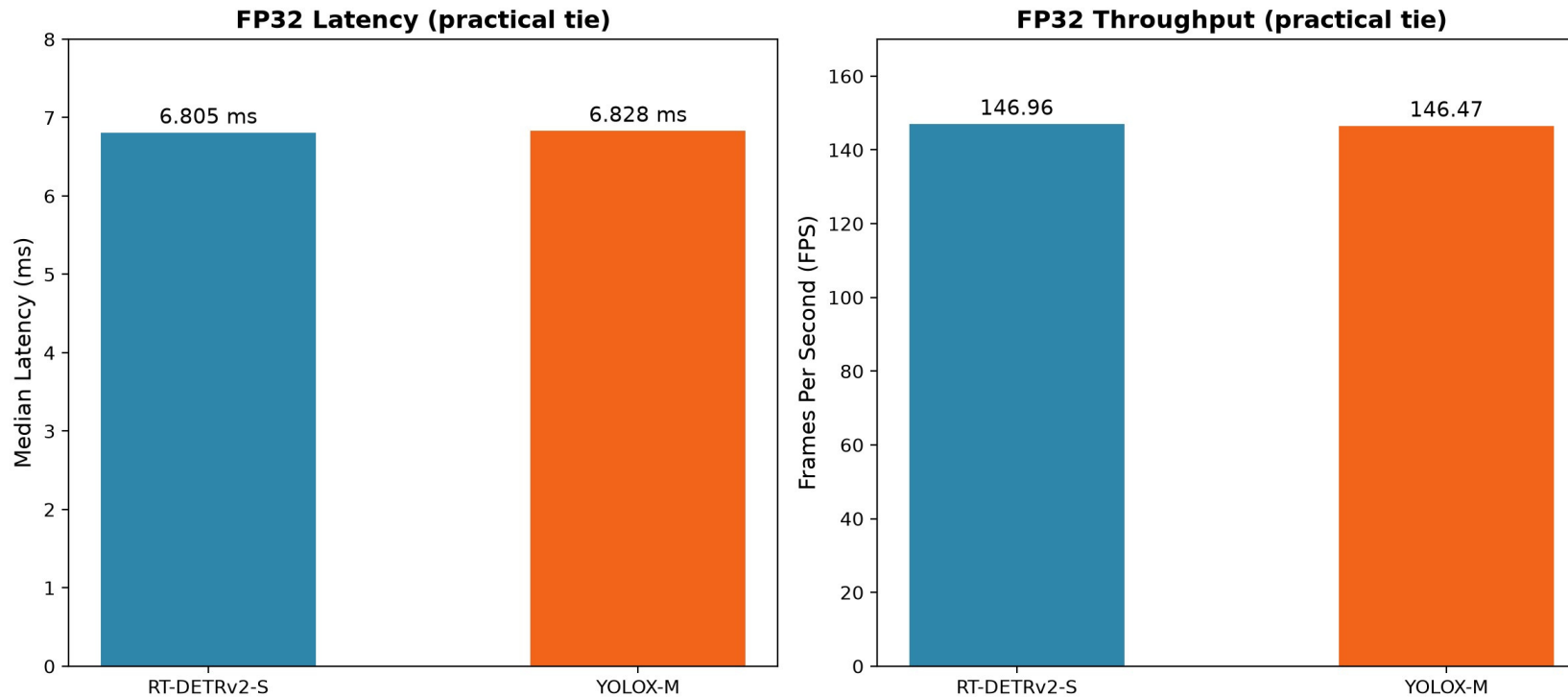
Batch=1, 640x640, RTX 5060 Ti, forward pass only

KEY TAKEAWAY

FP32 is effectively tied

- Both models run at about 147 FPS in FP32.
- This chart prevents overclaiming the FP16 result.
- Deployment should benchmark the exact precision used.

FP32 Inference Speed: RT-DETRv2-S vs YOLOX-M
(batch=1, 640×640, RTX 5060 Ti, forward-pass only)



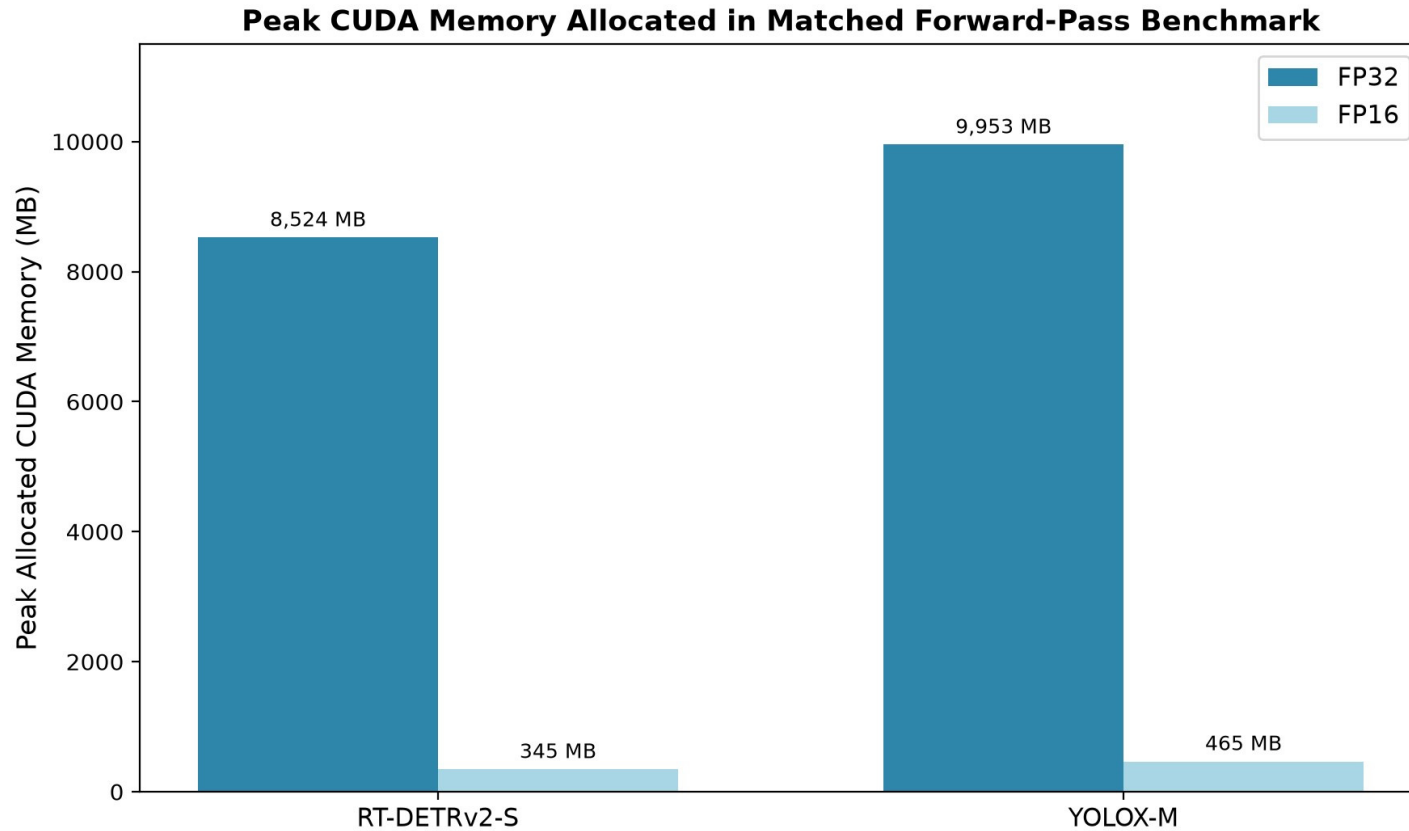
Peak CUDA memory allocated

Forward-pass benchmark, not full deployment memory

KEY TAKEAWAY

FP16 memory is far lower

- FP16 allocation is much lower than FP32 in this benchmark.
- The chart is a forward-pass measurement, not total deployment footprint.
- Use memory data alongside speed and accuracy.



Note: batch=1, 640x640, forward-pass only — not a general deployment memory requirement.

Different strengths, different trade-offs.

Controlled practical benchmark - not a universal architecture verdict

RT-DETRv2-S delivered the stronger accuracy and model-size trade-off in this benchmark.

YOLOX-M retained a modest FP16 throughput advantage, making the speed/accuracy trade-off deployment-dependent.

What we learned

- mAP alone is not enough - class-wise behavior and object scale reveal why models differ.
- RT-DETRv2-S converged much earlier in this setup, while YOLOX-M peaked later.
- Model size, FP16 speed, FP32 speed and memory should be evaluated together for practical decisions.
- Same dataset, same evaluation framing, and transparent hardware benchmarking make the comparison reproducible.

Code + results: github.com/NishchayChaudhary/traffic-nature-13-benchmark